

BrainerX — CompTIA Security+

Chapter 0 : Introduction to Cybersecurity

Prerequisites: Basic networking knowledge, familiarity with Linux terminal

Tools Required: Linux/Kali Linux, Wireshark, Nmap, Browser with DevTools

Table of Contents

#	Module
1	Introduction to Cybersecurity
2	The OSI Model — Layers and Protocols
3	The TCP/IP Model — Practical View
5	DNS — Domain Name System
6	HTTP, HTTPS, and the SSL/TLS Handshake
—	Review, Q&A, and Wrap-Up

Module 1 — Introduction to Cybersecurity

1.1 What is Cybersecurity?

Cybersecurity is the practice of protecting systems, networks, programs, and data from digital attacks, unauthorized access, damage, or destruction. It encompasses technologies, processes, and practices designed to defend computers, servers, mobile devices, electronic systems, and data.

Modern cybersecurity operates across several domains:

- **Network Security** — protecting the infrastructure
- **Application Security** — securing software and apps
- **Information Security** — protecting data integrity and privacy
- **Operational Security** — processes and decisions for handling data
- **Endpoint Security** — securing individual devices
- **Cloud Security** — protecting cloud-based assets

1.2 The CIA Triad

The **CIA Triad** is the foundational model for information security:

The Foundation of Cybersecurity: CIA

Confidentiality, integrity, and availability - the CIA triad



29 Information Classification: PUBLIC
© 2024 Natsar, LLC. All rights reserved.

Principle	Definition	Attack that violates it
Confidentiality	Data is accessible only to authorized parties	Eavesdropping, data breach, MITM
Integrity	Data is accurate and has not been tampered with	Man-in-the-Middle, data injection
Availability	Systems and data are accessible when needed	Denial of Service (DoS), ransomware

1.3 Types of Threats and Attackers

Threat Categories

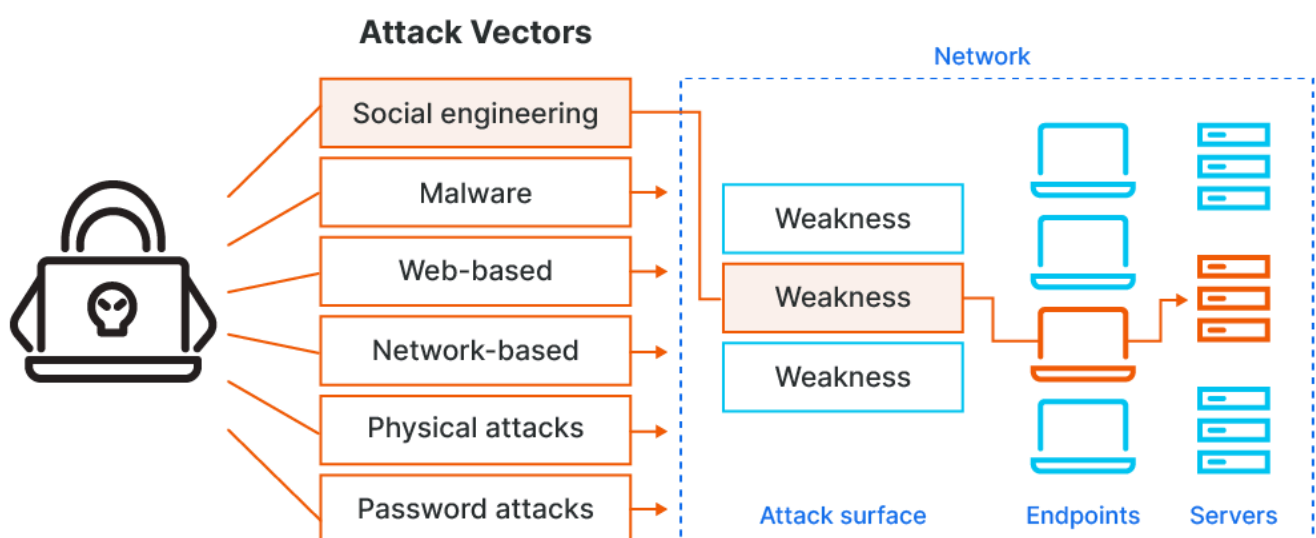
Threat Category	Subcategory Type	Examples
External Threats	Threat Actors	Cybercriminals, Nation States, Hacktivists
Internal Threats	Human/Internal Sources	Malicious Insider, Negligent Employee
Technological Threats	System Weaknesses	Vulnerabilities, Misconfigurations, Unpatched Systems
Supply Chain Threats	Third-Party Risks	Compromised vendors, infected software updates



Types of Attackers (Threat Actors)

- ◦ **White Hat** — Ethical hackers, authorized penetration testers
- ◦ **Black Hat** — Malicious attackers with criminal intent
- ◦ **Grey Hat** — Find vulnerabilities without permission, may or may not cause harm
- ◦ **Nation-State** — Government-sponsored cyber warfare units
- ◦ **Cybercriminals** — Financially motivated organized groups
- ◦ **Script Kiddies** — Inexperienced users using pre-built tools

1.4 Attack Vectors in Cyber Security



What is an Attack Vector?

An attack vector is the **path or method used by an attacker to gain unauthorized access** to systems, networks, or applications.

For example:

- An attacker may exploit **unpatched software** to compromise a system.
- Or deploy **malware** to infect a device.
- In cloud environments, attackers often use **phishing or social engineering** to trick users into giving access.

These vectors represent the “entry points” into a target system.

What is the Difference Between an Attack Vector vs Vulnerability?

These two concepts are related but fundamentally different:

Concept	Definition	Example
Attack Vector	The method or path used by an attacker to exploit a system	Phishing email, malware, malicious link
Vulnerability	A weakness in software, hardware, or configuration that can be exploited	Weak password, unpatched system, coding bug

In simple terms:

- **Vulnerability = weakness**
- **Attack Vector = way to exploit that weakness**

What is an Attack Surface? How does it differ from Attack Vectors?

The relationship can be understood like this:

Term	Meaning
Attack Vector	The specific method used to attack
Attack Surface	The total set of all possible entry points

Explanation:

- Attack vectors are the **individual techniques or paths**
- Attack surface is the **sum of all possible vectors**

Example:

- Open ports
- Web applications

- User accounts
- APIs
- Physical access points

Reducing the attack surface means reducing the number of possible attack vectors.

Threat Vectors vs Attack Vectors

Although often used interchangeably, there is a subtle difference:

Term	Meaning
Attack Vector	Specific method used to execute an attack
Threat Vector	Broader category or route of potential threats

Threat vectors include:

- Insider threats
- Environmental risks
- Organizational weaknesses

Attack vectors are the **technical execution methods** inside those threats.

What are Attack Vectors in Cyber Security?

Attack vectors are the **methods attackers use to breach systems**, including:

- Human manipulation (phishing)
 - Software exploitation (SQL injection)
 - Network interception (MITM)
 - Malware delivery
 - Physical intrusion
-

What is the Most Common Attack Vector?

One of the most common attack vectors is:

Phishing

Phishing uses fake messages or websites to trick users into:

- Revealing passwords
- Sharing sensitive data
- Clicking malicious links

It remains the most effective because it targets **human behavior instead of technology**.

Attack Vector Types and Categories

1. Social Engineering Attack Vectors

These exploit human behavior rather than technical flaws:

Type	Description	Example
Phishing	Fake messages to steal data	Fake bank email
Pretexting	Fake scenario to extract info	Fake IT support call
Baiting	Offering something to lure victim	Free USB or download
Tailgating	Following into restricted area	Physical access abuse

2. Malware-Based Attack Vectors

Software designed to harm or exploit systems:

Type	Description
Virus	Infects files and spreads
Worm	Self-replicates across networks
Trojan	Hidden malicious software
Infostealer	Steals credentials/data
Ransomware	Encrypts files for ransom

3. Web-Based Attack Vectors

These target web applications:

Type	Description
SQL Injection	Injects malicious SQL queries
XSS	Injects malicious scripts into web pages
CSRF	Forces users to perform unwanted actions

4. Network-Based Attack Vectors

These exploit network communication:

Type	Description
MITM	Intercepts communication

Type	Description
DoS	Overloads a system
DDoS	Distributed overload attack

5. Physical Attack Vectors

Direct physical access to systems:

Type	Description
Device Theft	Stealing laptops or phones
Tampering	Physically modifying hardware

6. Credential-Based Attack Vectors

Attacks targeting authentication systems:

Type	Description
Brute Force	Trying all password combinations
Dictionary Attack	Using common passwords list
Credential Stuffing	Reusing leaked credentials

7. Cryptographic / Initialization Vector Attacks

These target encryption mechanisms:

- Replay attacks (reusing intercepted data)
 - Chosen-plaintext attacks
 - Bit-flipping attacks
-

8. Multi-Vector Attacks

A multi-vector attack uses **multiple attack methods at the same time or sequentially**.

Example:

- Phishing + Malware + Credential theft combined

Advanced persistent threats (APTs) often use this approach.

Summary

- Attack vectors = **how attackers get in**

- Vulnerabilities = **what they exploit**
- Attack surface = **all possible entry points**
- Threat vectors = **broader categories of risk paths**

Reducing both attack vectors and attack surface is essential for strong cybersecurity defense.

1.5 The Attack Lifecycle (Cyber Kill Chain)

The **Lockheed Martin Cyber Kill Chain** describes the stages of a cyberattack:

```
[1] Reconnaissance
    ↓
[2] Weaponization
    ↓
[3] Delivery
    ↓
[4] Exploitation
    ↓
[5] Installation
    ↓
[6] Command & Control (C2)
    ↓
[7] Actions on Objectives
```

Understanding this model helps defenders identify **where** and **when** to intervene.

Cyber Kill Chain

The 7 Stages of a Cyber Attack



 Attackers achieve their goals like data theft or disruption.



Module 1 — Review Questions

- Q1. Define the three components of the CIA Triad and give a real-world example for each.
- Q2. A hospital's patient database is encrypted by ransomware and becomes inaccessible. Which principle(s) of the CIA Triad are violated?
- Q3. What is the difference between a white hat and a grey hat hacker? Give an example scenario for each.
- Q4. In which phase of the Kill Chain does Nmap scanning fit? Justify your answer.

Module 2 — The OSI Model (30 min)

2.1 Why Do We Need a Network Model?

Before the OSI model, different vendors built incompatible networking equipment. The **Open Systems Interconnection (OSI) model**, developed by the ISO in 1984, created a universal framework for how network communication should work — allowing devices from different manufacturers to communicate.

2.2 The 7 Layers of the OSI Model

#	Layer Name	Function
7	Application	End-user interface (HTTP, FTP, SMTP, DNS)
6	Presentation	Data format, encryption, compression (SSL)
5	Session	Session management, authentication
4	Transport	End-to-end delivery, TCP/UDP, ports
3	Network	Routing, IP addressing, ICMP
2	Data Link	MAC addressing, framing, error detection
1	Physical	Cables, switches, bits, electrical signals

Mnemonic (Top-Down): "All People Seem To Need Data Processing"

2.3 Layer-by-Layer Deep Dive

Layer 7 — Application Layer

- The **only layer** the end user directly interacts with

- Protocols: **HTTP**, **HTTPS**, **FTP**, **SMTP**, **POP3**, **IMAP**, **DNS**, **SNMP**, **SSH**, **Telnet**
- **PDU (Protocol Data Unit):** Data / Message

Layer 6 — Presentation Layer

- Handles **encoding, encryption, and compression**
- Translates data between application and network format
- Protocols/Standards: **SSL/TLS**, **JPEG**, **MPEG**, **ASCII**, **UTF-8**
- **PDU:** Data

Layer 5 — Session Layer

- Manages **sessions** (establishment, maintenance, termination)
- Handles **authentication** and **authorization**
- Protocols: **NetBIOS**, **RPC**, **PPTP**
- **PDU:** Data

Layer 4 — Transport Layer

- Provides **end-to-end** communication between processes
- **Port numbers** are defined at this layer
- **Flow control, error recovery, segmentation**
- Key protocols:
 - **TCP** (Transmission Control Protocol) — reliable, connection-oriented
 - **UDP** (User Datagram Protocol) — unreliable, connectionless, fast
- **PDU:** Segment (TCP) / Datagram (UDP)

Layer 3 — Network Layer

- Responsible for **logical addressing** and **routing**
- Determines the best path for data across networks
- Protocols: **IP** (**IPv4/IPv6**), **ICMP**, **ARP**, **OSPF**, **BGP**
- **PDU:** Packet

Layer 2 — Data Link Layer

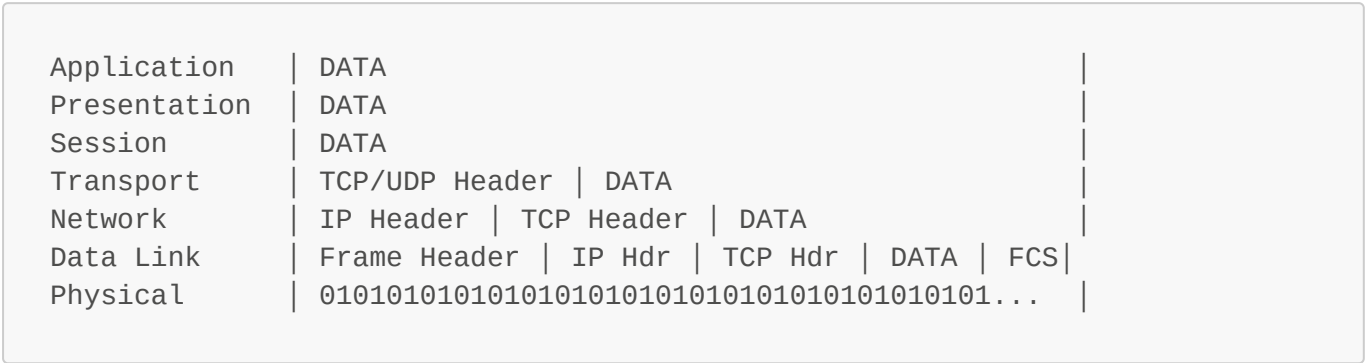
- Handles **physical addressing** (MAC address)
- Detects and corrects errors from Layer 1
- Two sublayers: **LLC** (Logical Link Control) and **MAC** (Media Access Control)
- Protocols: **Ethernet**, **Wi-Fi** (**802.11**), **PPP**, **ARP**
- **PDU:** Frame

Layer 1 — Physical Layer

- Transmits raw **bits** over physical media
- Deals with: cables, fiber optics, radio frequencies, voltages
- Protocols/Standards: **USB**, **RS-232**, **Ethernet** (**physical**), **DSL**, **Bluetooth**
- **PDU:** Bit

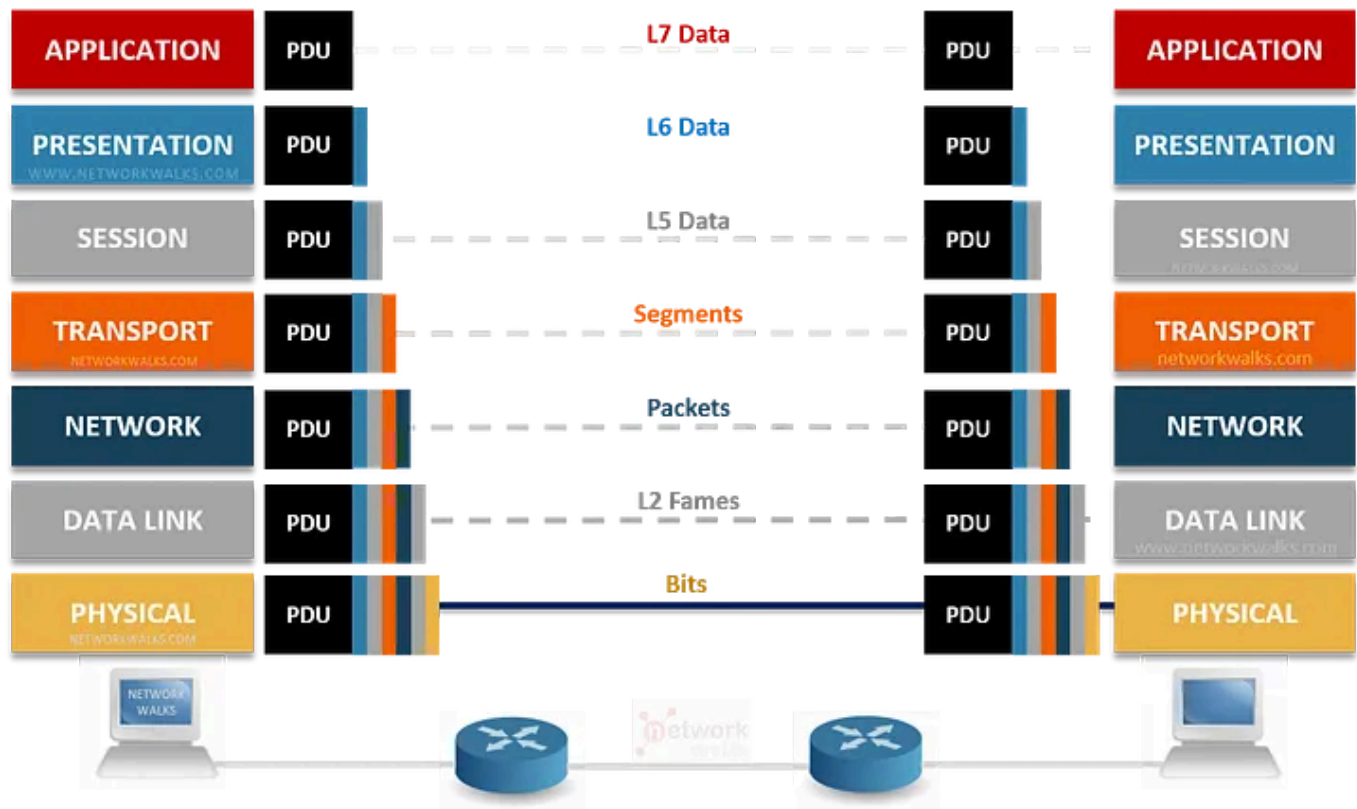
2.4 Data Encapsulation

As data travels **down** the OSI stack, each layer **wraps** the data with its own header (and sometimes trailer). This process is called **encapsulation**. The reverse happens on the receiving side (**decapsulation**).



2.5 OSI vs Real-World — Where Attacks Happen

Layer	Common Attack
Application (7)	SQL Injection, XSS, HTTP Flood
Presentation (6)	SSL Stripping
Session (5)	Session Hijacking
Transport (4)	SYN Flood, Port Scanning
Network (3)	IP Spoofing, ICMP Flood
Data Link (2)	ARP Spoofing, MAC Flooding
Physical (1)	Cable tapping, Physical intrusion



Module 2 — Review Questions

Q1. At which OSI layer does a **router** primarily operate? What about a **switch**? A **hub**?

Q2. A packet has a source IP address of **192.168.1.10** and destination IP **8.8.8.8**. At which OSI layer is this address information processed?

Q3. Why is the Transport Layer important for cybersecurity? What specific information does it expose to an attacker?

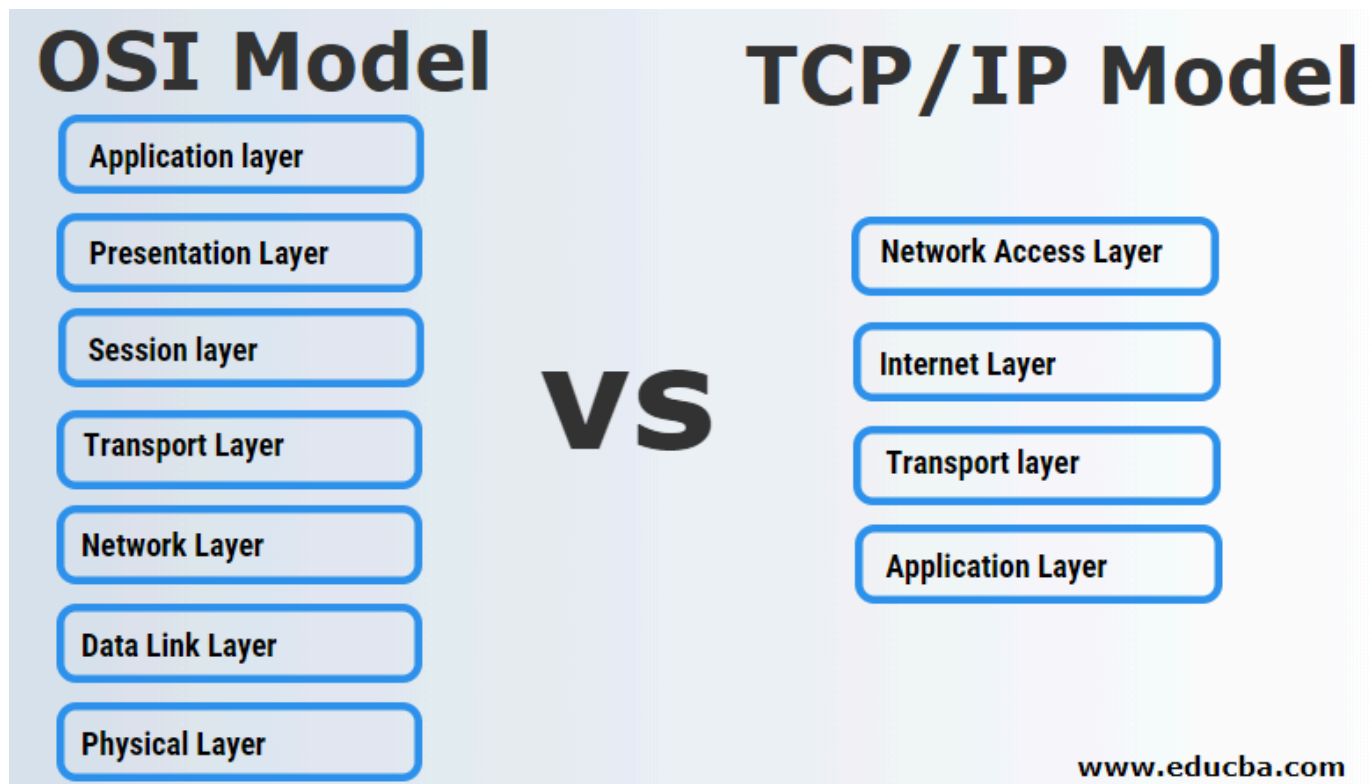
Q4. Match each protocol to its OSI layer: **HTTP**, **IP**, **TCP**, **SSL**, **Ethernet**, **DNS**, **ARP**.

Q5. When a web browser sends a request to a web server, describe the encapsulation that occurs across the OSI layers.

Module 3 — The TCP/IP Model

3.1 TCP/IP vs OSI

The **TCP/IP model** (also called the **DoD model**) is the practical implementation used on the Internet. It collapses the 7-layer OSI model into **4 layers**:



3.2 IPv4 Addressing

An **IPv4 address** is a 32-bit number written as four decimal octets separated by dots.

Example: 192.168.1.100
 Binary: 11000000.10101000.00000001.01100100

IP Address Classes

Class	Range	Default Subnet Mask	Usage
A	1.0.0.0 – 126.255.255.255	255.0.0.0 /8	Large networks
B	128.0.0.0 – 191.255.255.255	255.255.0.0 /16	Medium networks
C	192.0.0.0 – 223.255.255.255	255.255.255.0 /24	Small networks

Private IP Ranges (RFC 1918)

Range	CIDR
10.0.0.0 – 10.255.255.255	10.0.0.0/8
172.16.0.0 – 172.31.255.255	172.16.0.0/12
192.168.0.0 – 192.168.255.255	192.168.0.0/16

Security relevance: Attackers scan private ranges during internal reconnaissance. Understanding CIDR notation is essential for interpreting Nmap scan ranges.

3.3 IPv6 Addressing

IPv6 was introduced to address **IPv4 exhaustion**. It uses **128-bit** addresses:

```
Example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334
Short:   2001:db8:85a3::8a2e:370:7334
```

IPv6 provides approximately **3.4×10^{38}** addresses — a practically unlimited pool.

3.4 Useful Network Commands

```
# Display your IP address and network interfaces
ip addr show
ifconfig          # older systems

# Test reachability (uses ICMP Echo Request)
ping 8.8.8.8
ping6 ::1         # IPv6 loopback test

# Trace the path to a host
traceroute 8.8.8.8
traceroute6 ipv6.google.com

# Display routing table
ip route show
route -n

# Show ARP table (IP → MAC mappings)
arp -a

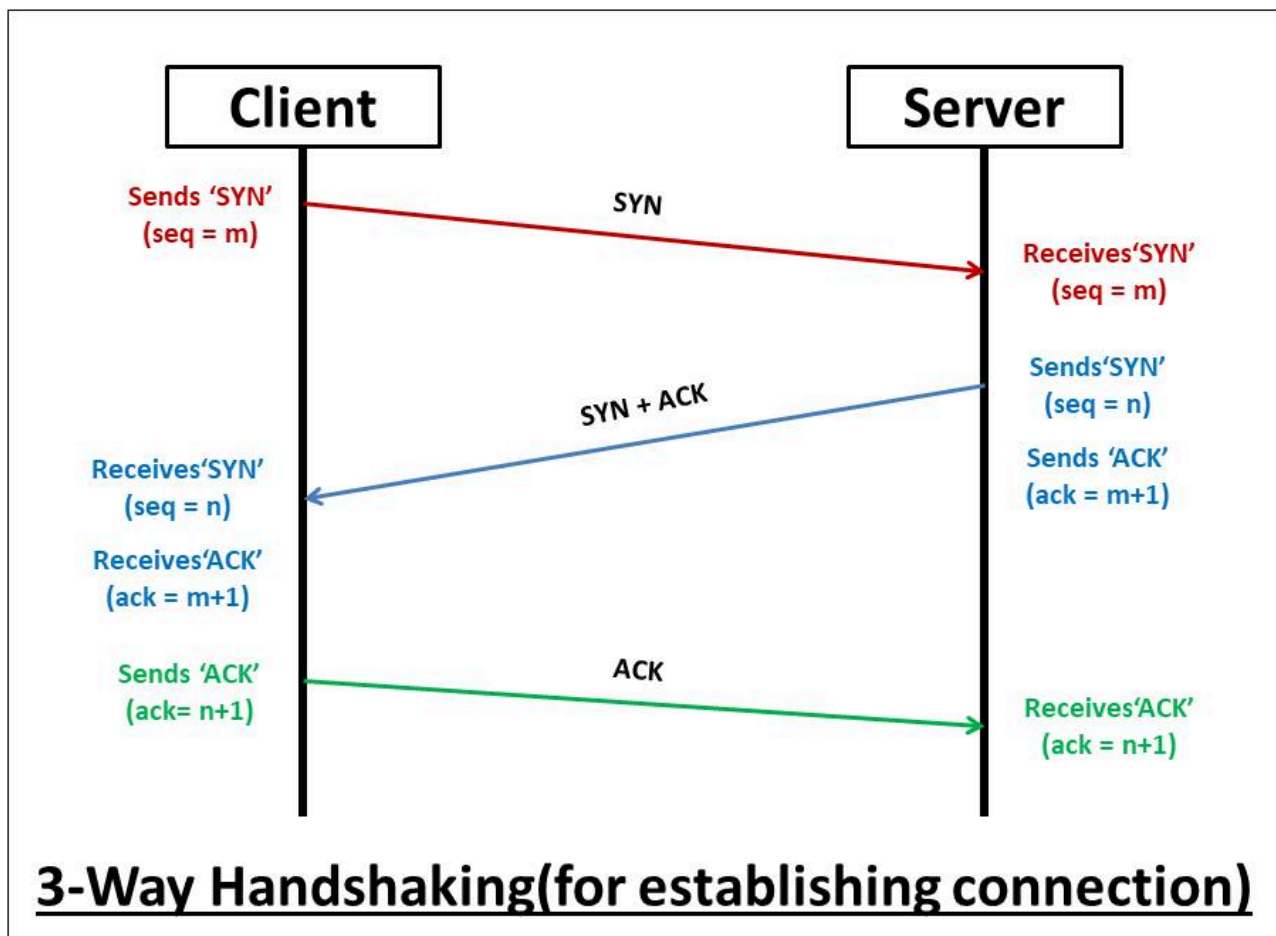
# Show open ports and connections
netstat -tuln
ss -tuln

# DNS lookup
nslookup google.com
dig google.com
host google.com

# Display network interface statistics
ip -s link
```

3.5 The TCP Three-Way Handshake

Before any data can be exchanged over TCP, a **connection** must be established using a **three-way handshake**:



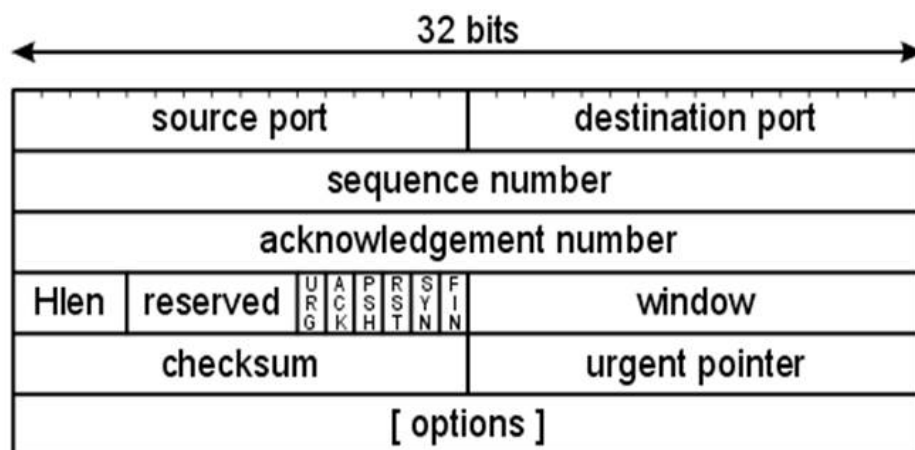
TCP Flags and their meanings:

Flag	Meaning	Security Relevance
SYN	Synchronize — initiate connection	SYN Flood (DoS)
ACK	Acknowledge — confirm receipt	Used to map firewall rules
FIN	Finish — terminate connection	Used in stealthy scans
RST	Reset — abort connection	Port closed response
PSH	Push — send data immediately	Data delivery
URG	Urgent pointer	Rarely used

Nmap exploits TCP flag behavior. A **SYN scan** (`-ss`) sends a SYN and analyzes the response — **SYN-ACK** means open, **RST** means closed.

3.6 TCP vs UDP — Comparison

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Guaranteed delivery	Best effort
Ordering	In-order delivery	No ordering
Speed	Slower	Faster
Error recovery	Yes	No
Use cases	HTTP, HTTPS, SSH, FTP, SMTP	DNS, VoIP, Video Streaming, DHCP
Header size	20–60 bytes	8 bytes



TCP Header

Module 3 — Review Questions

Q1. What is the purpose of the three-way handshake? What happens if the last ACK is never sent?

Q2. A packet capture shows a host sending SYN packets to 1000 different ports on a target machine in 2 seconds with no completed handshakes. What type of scan or attack does this suggest?

Q3. Why would an attacker prefer UDP scanning? What makes it harder to perform than TCP scanning?

- Q4. Convert the IP address 192.168.10.50 to binary. What is the network address if the subnet mask is /24?
- Q5. What is the difference between a public IP and a private IP? Why can't a private IP be routed on the Internet?

Hour 2 — Core Protocols and Cryptography

Module 4 — Core Protocols: TCP, UDP, ICMP

4.1 Port Numbers — The Language of Services

Ports are **16-bit numbers** (0–65535) that identify specific processes or services:

Range	Type	Description
0 – 1023	Well-Known Ports	Reserved for standard services
1024 – 49151	Registered Ports	Registered by software vendors
49152 – 65535	Dynamic/Ephemeral Ports	Temporary, used by clients

Critical Well-Known Ports (Must Memorize)

Port	Protocol	Service
20, 21	TCP	FTP (data, control)
22	TCP	SSH
23	TCP	Telnet (insecure!)
25	TCP	SMTP (email sending)
53	TCP/UDP	DNS
67, 68	UDP	DHCP
80	TCP	HTTP
110	TCP	POP3
143	TCP	IMAP
443	TCP	HTTPS
3306	TCP	MySQL
3389	TCP	RDP (Remote Desktop)
8080	TCP	HTTP alternative

Security insight: Open ports are an attack surface. An attacker scans ports to identify running services and match them to known vulnerabilities.

4.2 ICMP — Internet Control Message Protocol

ICMP is used for **error reporting and diagnostics**. It operates at the **Network Layer (Layer 3)** and has **no port numbers**.

Key ICMP Message Types

Type	Code	Meaning
0	0	Echo Reply (ping response)
3	0–15	Destination Unreachable
5	—	Redirect
8	0	Echo Request (ping)
11	0	Time Exceeded (used by traceroute)

The image shows a Wireshark packet capture of an ICMP Echo (ping) request and reply. The packet list shows four packets: a request from 10.10.72.157 to 74.125.193.99 (ID 0x1bfc, seq 2/512), a reply from 74.125.193.99 to 10.10.72.157 (ID 0x1bfc, seq 2/512), a request from 10.10.72.157 to 74.125.193.99 (ID 0x1bfc, seq 3/768), and a reply from 74.125.193.99 to 10.10.72.157 (ID 0x1bfc, seq 3/768). The packet details pane for the first packet shows the Ethernet II header, followed by the Internet Protocol Version 4 header (Source: 10.10.72.157, Destination: 74.125.193.99), and the ICMP (1) header (Type: Echo (ping) request, Code: 0, Identifier: 0x1bfc, Sequence: 2/512). The ICMP header is highlighted with a red box.

```
# Ping — send ICMP Echo Request
ping -c 4 8.8.8.8

# Ping with specific packet size
ping -s 1400 192.168.1.1

# Traceroute — exploits ICMP TTL Exceeded messages
traceroute 8.8.8.8
```

Security use: **ping sweep** (nmap -sP) uses ICMP to discover live hosts. Firewalls often **block ICMP** to prevent host discovery — this is why Nmap sometimes uses TCP/SYN pings as alternatives.

4.3 ARP — Address Resolution Protocol

ARP maps an IP address to a **MAC address** on the local network (Layer 2–3 boundary).

Who has 192.168.1.1? (ARP Request – broadcast)

→ FF:FF:FF:FF:FF:FF

192.168.1.1 is at AA:BB:CC:DD:EE:FF (ARP Reply – unicast)

←

No.	Time	Source	Destination	Protocol	Length	Info
69	3.483300659	HP_cc:d6:b9	Broadcast	ARP	60	Who has 10.143.90.167? Tell 10.143.90.23
122	5.517743097	Cisco_27:d3:69	Broadcast	ARP	60	Who has 10.143.90.14? Tell 10.143.90.102
161	8.530076099	Cisco_27:d3:69	Broadcast	ARP	60	Who has 10.143.90.14? Tell 10.143.90.102
176	9.817521713	HP_cc:d6:b9	Broadcast	ARP	60	Who has 10.143.90.167? Tell 10.143.90.23
180	10.474630480	HP_cc:d6:b9	Broadcast	ARP	60	Who has 10.143.90.167? Tell 10.143.90.23
187	11.483681187	HP_cc:d6:b9	Broadcast	ARP	60	Who has 10.143.90.167? Tell 10.143.90.23
191	12.546492084	Cisco_27:d3:69	Broadcast	ARP	60	Who has 10.143.90.14? Tell 10.143.90.102
291	20.565873141	Cisco_27:d3:69	Broadcast	ARP	60	Who has 10.143.90.14? Tell 10.143.90.102
613	45.626446961	Cisco_27:d3:69	Broadcast	ARP	60	Who has 10.143.90.123? Tell 10.143.90.102
633	48.633902902	Cisco_27:d3:69	Broadcast	ARP	60	Who has 10.143.90.123? Tell 10.143.90.102
675	52.634044505	Cisco_27:d3:69	Broadcast	ARP	60	Who has 10.143.90.123? Tell 10.143.90.102

▶ Frame 32: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface eth0, id 0
 ▶ Ethernet II, Src: HP_cc:d6:b9 (e0:70:ea:cc:d6:b9), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
 ▶ Address Resolution Protocol (request)

```
# View ARP cache
arp -a

# Send ARP request manually
arping 192.168.1.1
```

ARP Spoofing: An attacker sends fake ARP replies to associate their MAC with the gateway IP — allowing **Man-in-the-Middle** attacks. This is a critical Layer 2 attack vector.

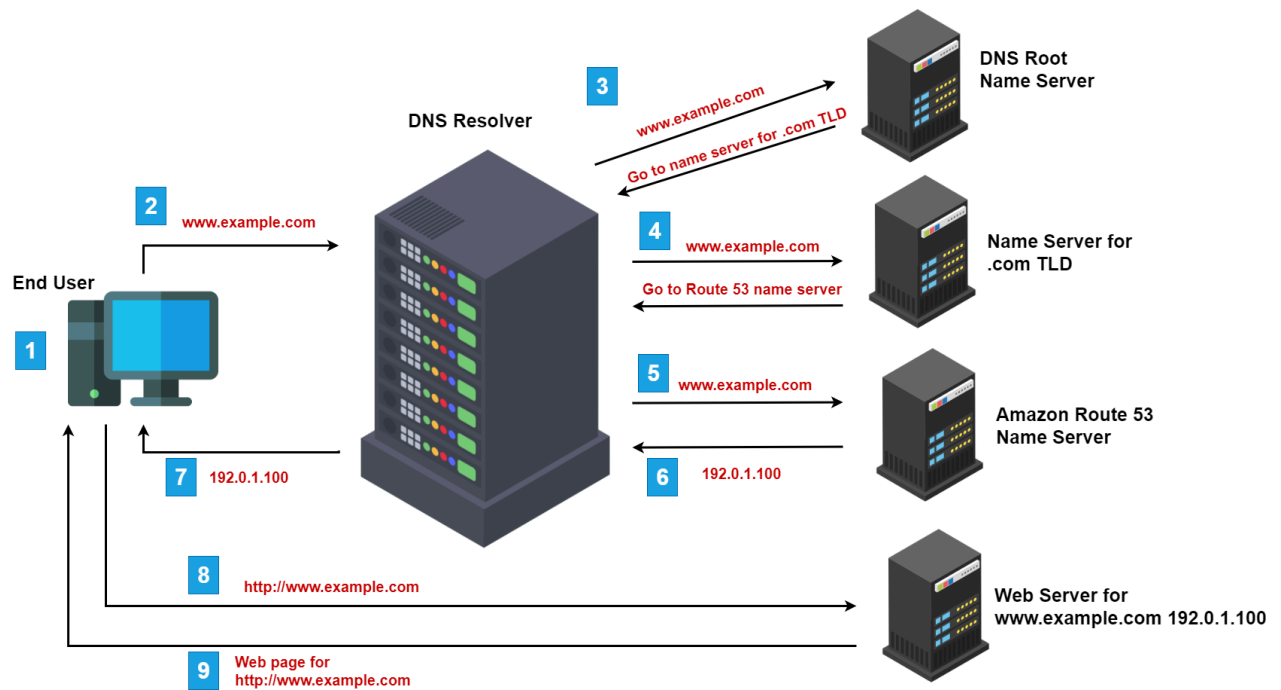
Module 5 — DNS — Domain Name System

5.1 What is DNS?

DNS (Domain Name System) is the "phone book" of the Internet. It translates human-readable **domain names** (e.g., **google.com**) into **IP addresses** (e.g., **142.250.185.46**).

- Operates on **port 53**
- Uses **UDP** for queries (≤512 bytes), **TCP** for zone transfers and large responses

- Hierarchical and distributed architecture



5.2 DNS Resolution Process

When you type `www.google.com` in your browser:

The diagram illustrates the recursive DNS resolution process for the domain `www.google.com`. It shows the flow of queries and responses between four levels of the DNS hierarchy: Browser, Auth NS, Resolver, and Root NS, with TLD NS also shown.

Participants:

- Browser:** Initiates the query.
- Auth NS:** The authoritative name server for the domain.
- Resolver:** The local DNS resolver.
- Root NS:** The root of the DNS hierarchy.
- TLD NS:** The top-level domain name server.

Sequence of Events:

- The **Browser** sends a query for `"www.google.com?"` to the **Resolver**.
- The **Resolver** sends a query for `"google.com?"` to the **Root NS**.
- The **Root NS** responds with `".com NS is X"` to the **Resolver**.
- The **Resolver** sends a query for `"google.com?"` to the **TLD NS**.
- The **TLD NS** responds with `"google.com NS is Y"` to the **Resolver**.
- The **Resolver** sends a query for `"www.google.com?"` to the **Auth NS**.
- The **Auth NS** responds with `"142.250.185.46"` to the **Resolver**.
- The **Resolver** sends the final IP address `"142.250.185.46"` back to the **Browser**.

Resolution Chain:

1. **Local Cache** → check browser/OS cache
2. **Recursive Resolver** → your ISP or configured DNS (e.g., 8.8.8.8)
3. **Root Name Server** → 13 root server clusters (labeled A–M)

4. **TLD Name Server** → responsible for **.com**, **.org**, **.dz**, etc.

5. **Authoritative Name Server** → the final answer

5.3 DNS Record Types

Record	Full Name	Purpose	Example
A	Address	Maps hostname → IPv4	google.com → 142.250.185.46
AAAA	IPv6 Address	Maps hostname → IPv6	google.com → 2607:f8b0::
MX	Mail Exchange	Mail server for domain	google.com → smtp.google.com
CNAME	Canonical Name	Alias for another domain	www.google.com → google.com
NS	Name Server	Authoritative DNS servers	google.com → ns1.google.com
TXT	Text	SPF, DKIM, verification	"v=spf1 include:..."
PTR	Pointer	Reverse DNS (IP → name)	8.8.8.8 → dns.google
SOA	Start of Authority	Zone administration info	—

5.4 DNS Commands

```
# Basic DNS lookup (A record)
nslookup google.com

# Query specific record type
dig google.com MX
dig google.com NS
dig google.com TXT
dig google.com AAAA

# Query a specific DNS server
dig @8.8.8.8 google.com

# Reverse DNS lookup (PTR record)
dig -x 8.8.8.8

# Full DNS resolution trace
dig +trace google.com

# Query all record types
dig google.com ANY

# Simple host lookup
host google.com
host -t MX google.com
```

```
# Test DNS on Windows
nslookup -type=MX google.com 8.8.8.8
```

5.5 DNS Security Issues

DNS Attacks

Attack	Description
DNS Spoofing / Cache Poisoning	Injecting false DNS records into a resolver's cache
DNS Hijacking	Redirecting DNS queries to a malicious server
DNS Amplification (DDoS)	Using open resolvers to amplify attack traffic
DNS Zone Transfer Attack	Unauthorized transfer of all DNS records (AXFR)
DNS Tunneling	Encoding data inside DNS queries/responses to bypass firewalls

```
# Attempt a zone transfer (only works if misconfigured)
dig @ns1.target.com target.com AXFR

# Check if a server allows recursive queries (amplification risk)
dig @target-dns.com google.com
```

Warning: Only perform these on domains/servers you have explicit permission to test.

DNS Security Extensions (DNSSEC)

DNSSEC adds cryptographic signatures to DNS records to verify authenticity and prevent spoofing.

Module 5 — Review Questions

- Q1.** What happens if a DNS server is unreachable? How does this affect web browsing even if the web server itself is working perfectly?
 - Q2.** What is the difference between a **recursive resolver** and an **authoritative name server**?
 - Q3.** Using **dig**, how would you find the mail server for the domain **univ-batna2.dz**? Write the exact command.
 - Q4.** Explain how **DNS cache poisoning** works. What is the impact on a user's machine?
 - Q5.** Why is a **DNS zone transfer** a security risk? What information can an attacker gain from it?
-

Module 6 — HTTP, HTTPS, and the SSL/TLS Handshake

6.1 HTTP — HyperText Transfer Protocol

HTTP is the foundation of data communication on the Web. It is a **stateless, text-based** request-response protocol operating at Layer 7.

HTTP Request Structure

Let's see the structure of an actual request.



HTTP requests have a line, headers, a new line (CR + LF) after that, and a body at the end.

The HTTP request line is the command you send to the server to indicate what resource you want to get or process. It consists of:

Request method (in our example “GET”); Request-URL (this is the resource path); HTTP version string.

Web browsers use URLs, but HTTP uses URIs to address the resources.

URL stands for “uniform resource locator” and it describes a full unique address for a resource on the Internet, which consists of protocol + host + resource path, like in the example above.

URI stands for “uniform resource identifier” and it holds a full or relative unique path to a resource, for example “/about”.

When you request a resource over HTTP, you specify the relative URI of the resource in the request line and you specify the host name in the request headers. Both relative URI and host name come from the URL you want to access.

After the request line, the HTTP request headers are given. Headers specify specific parameters about the requested resource:

“Host” is an important header, holding the requested resource. If we have several Web sites on the same Web server (for example softuni.org and learn.softuni.org), this “Host” header will tell the server which website to access.

The other headers specify settings like what kind of content the client can accept and understands (for example only HTML or any content), what is the preferred language the client wants to use, what kind of compression the client understands (for example gzip and deflate), what are the client Web browser’s brand and version (encoded as the so-called “user agent” identifier) and other parameters.

The headers section in the HTTP request ends by an empty line (CR + LF twice).

After the request headers, comes the request body.

It can hold anything, for example, URL-encoded data or JSON objects or binary data.

In the given example the body is empty, which is typical for HTTP GET requests.

HTTP Request Methods

The HTTP request method defines what action will be performed on the identified resource. The most commonly used HTTP methods are GET, POST, PUT, DELETE and PATCH which correspond to read, create, update and delete (or CRUD) operations, respectively.

Idempotency and safety are properties of HTTP methods.

Safe methods can only be used for read-only operations since they do not alter the server state. Using GET or HEAD on a resource URL, for example, should never change the resource. Safe methods are considered GET, HEAD, TRACE and OPTIONS.

Idempotent methods can send multiple identical requests to the server and the outcome will always be the same and it does not matter how many times the requests will be sent. This does not mean, however, that the server has to respond in the same way to each request. For example, if we want to delete a resource we send a DELETE request. The first time the server returns a response that the file has been deleted. If you try to send the same request again the server will respond that the file has already been deleted. Here we have two different responses but the second request did not alter the server state. In this case, the DELETE operation is idempotent.

The following HTTP methods are idempotent: GET, HEAD, OPTIONS, TRACE, PUT and DELETE. All safe HTTP methods are idempotent. PUT and DELETE are idempotent but not safe.

You can see all methods and their function in the graphic below.

The main HTTP methods corresponding to the CRUD operations are POST, GET, PUT/PATCH and DELETE.

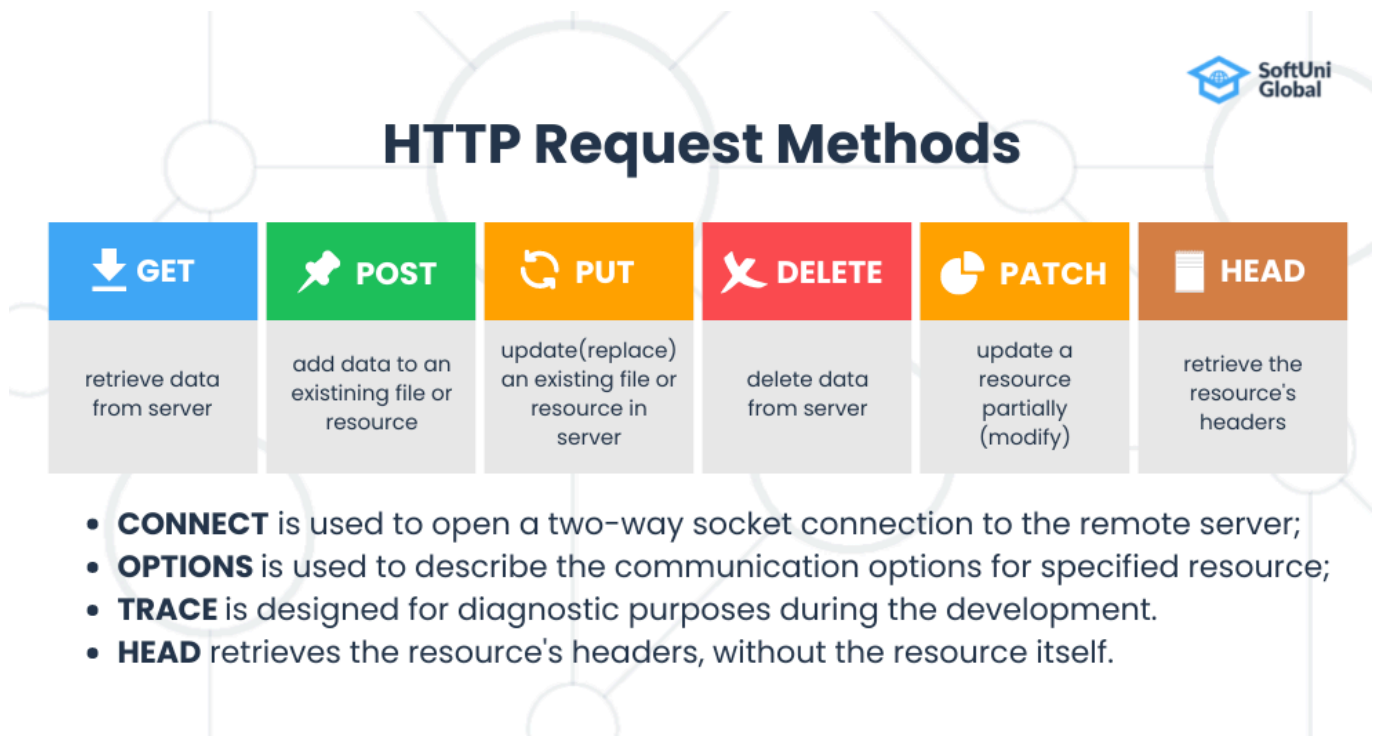
The GET method retrieves a specified resource (a list or a single resource). If there are no errors the method returns a representation of the resource in XML or JSON. GET is used to download a Web page, CSS file, script, document or other resources from a Web site. For example, a Web page’s content (fonts, images, etc.) is loaded using HTTP GET requests. This does not modify the state at the server-side, it only ‘reads’ it.

The POST method modifies the state of the server since it creates new resources. For example, when you login into a website, the login sends your credentials to the server using a POST request.

DELETE is used to delete (or remove) an existing resource. An example of an HTTP DELETE request is for deleting an item from the shopping cart in an e-commerce Web application.

The PATCH method updates an existing resource partially. It is used to modify a field of a given object. An example is an HTTP PATCH request for updating the quantity of an order item in the shopping cart in an e-commerce Web application.

The HTTP HEAD method retrieves the resource's headers, without the resource itself. HEAD is used rarely, for example, to check for modifications on the server-side.



HTTP Response Structure

After receiving and interpreting a request message, the server sends an HTTP response message. You can see an example below.



RESPONSE



The response message gives information on whether our request has been successfully executed or has failed. It consists of a status line, response headers and a response body.

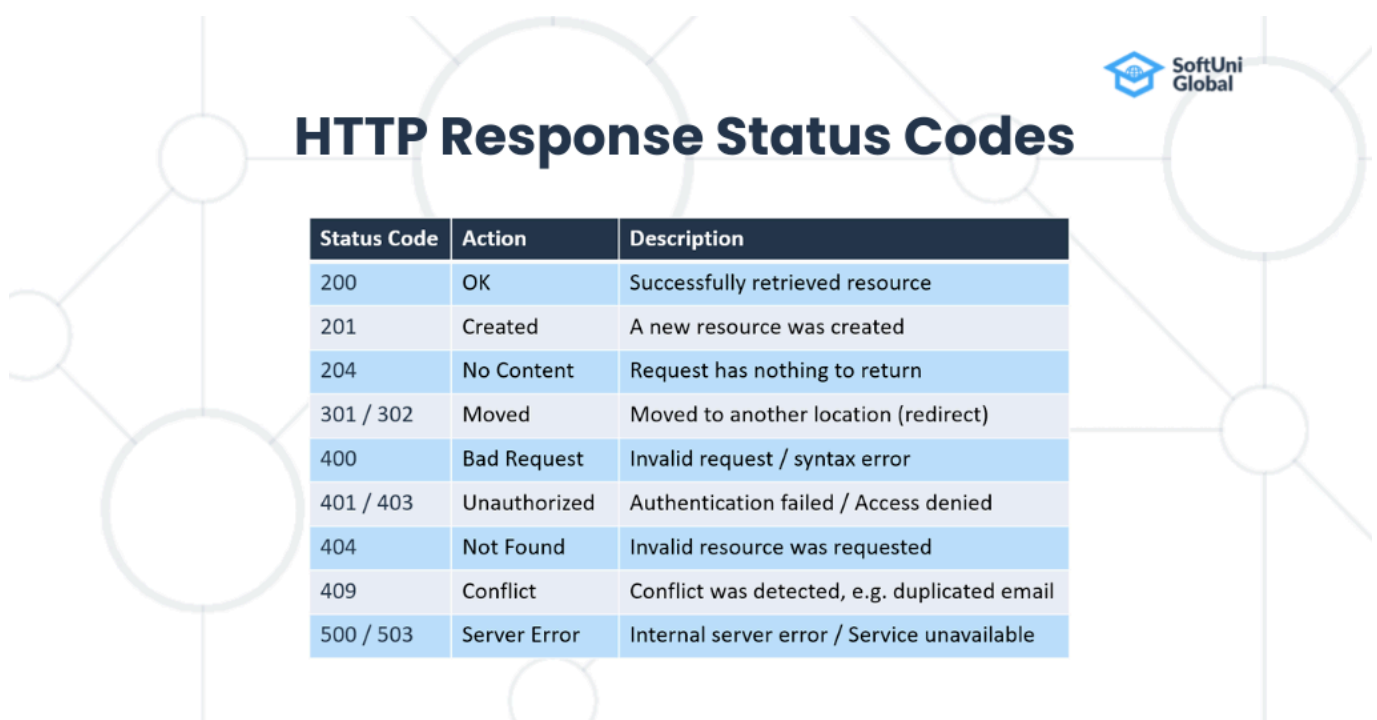
The HTTP response status line starts with the protocol version, followed by the response status code, followed by a human-readable text explanation of the status code.

The status code tells the client whether the requested operation was successful or not. It is a three-digit integer whose first digit defines the response class.

Status codes are:

Informational responses (100–199) Successful responses (200–299) Redirection messages (300–399) Client error responses (400–499) Server error responses (500–599)

You can check the graphic below to see the most common status codes.



Status Code	Action	Description
200	OK	Successfully retrieved resource
201	Created	A new resource was created
204	No Content	Request has nothing to return
301 / 302	Moved	Moved to another location (redirect)
400	Bad Request	Invalid request / syntax error
401 / 403	Unauthorized	Authentication failed / Access denied
404	Not Found	Invalid resource was requested
409	Conflict	Conflict was detected, e.g. duplicated email
500 / 503	Server Error	Internal server error / Service unavailable

After the HTTP status line come the HTTP response headers that provide metadata for the returned resource (or the returned error), such as content-encoding, content size in bytes, content last-modify date and many others.

After the response headers and the empty line separator, the HTTP response body comes. This is the requested resource that can be text, binary data or it can be empty. In the example we used above, the Web server returns a CSS script for styling a navigation bar.

Content-Type and Disposition Headers

HTTP headers play an important role in modern Web development.

The “Content-Type” and the “Content-Disposition” headers specify how to process the data in the HTTP request or in the HTTP response body. These headers can be used both in the HTTP requests and in the HTTP responses.

In the HTTP requests, the “Content-Type” header specifies what kind of data the client sends to the server, for example, a JSON document or URL-encoded form data or a plain-text document or a JPEG image. In the HTTP responses, the “Content-Type” header specifies what kind of data the server returns to the client, for example an HTML document or a JPEG image.

For example, the header “Content-Type: application/json” specifies a JSON-encoded data (a JSON object). By default, the UTF-8 encoding is used.

The “Content-Type: text/html; charset=utf-8” specifies an HTML document with UTF-8 encoding. Note that the encoding (or the charset) specified in the HTTP headers has a higher priority than the encoding specified in the header of the HTML document (using the “meta charset” HTML tag).

6.2 The Problem with HTTP

HTTP transmits data **in plaintext**. This means anyone between the client and server can read the data:

```
graph LR; You -- "[your data in clear text]" --> Web_Server[Web Server]; Attacker[Attacker intercepts here  
(Wi-Fi sniffing, MITM)] -.-> You;
```

The diagram illustrates a security vulnerability in HTTP. It shows a communication path from 'You' to a 'Web Server'. The data being sent is '[your data in clear text]'. An 'Attacker' is shown intercepting this data, with a note specifying '(Wi-Fi sniffing, MITM)'.

Never send passwords, credit card numbers, or sensitive data over plain HTTP.

6.3 HTTPS and TLS/SSL

HTTPS (HTTP Secure) is HTTP combined with **TLS (Transport Layer Security)** — the modern replacement for the older **SSL (Secure Sockets Layer)**.

HTTPS provides:

- **Encryption** — data is unreadable to interceptors
- **Authentication** — verifies the server's identity via certificates
- **Integrity** — ensures data was not modified in transit

Note: Despite common usage, SSL is deprecated. Modern systems use **TLS 1.2** or **TLS 1.3**. The terms "SSL" and "TLS" are often used interchangeably in everyday conversation.

The TLS 1.3 Handshake Process

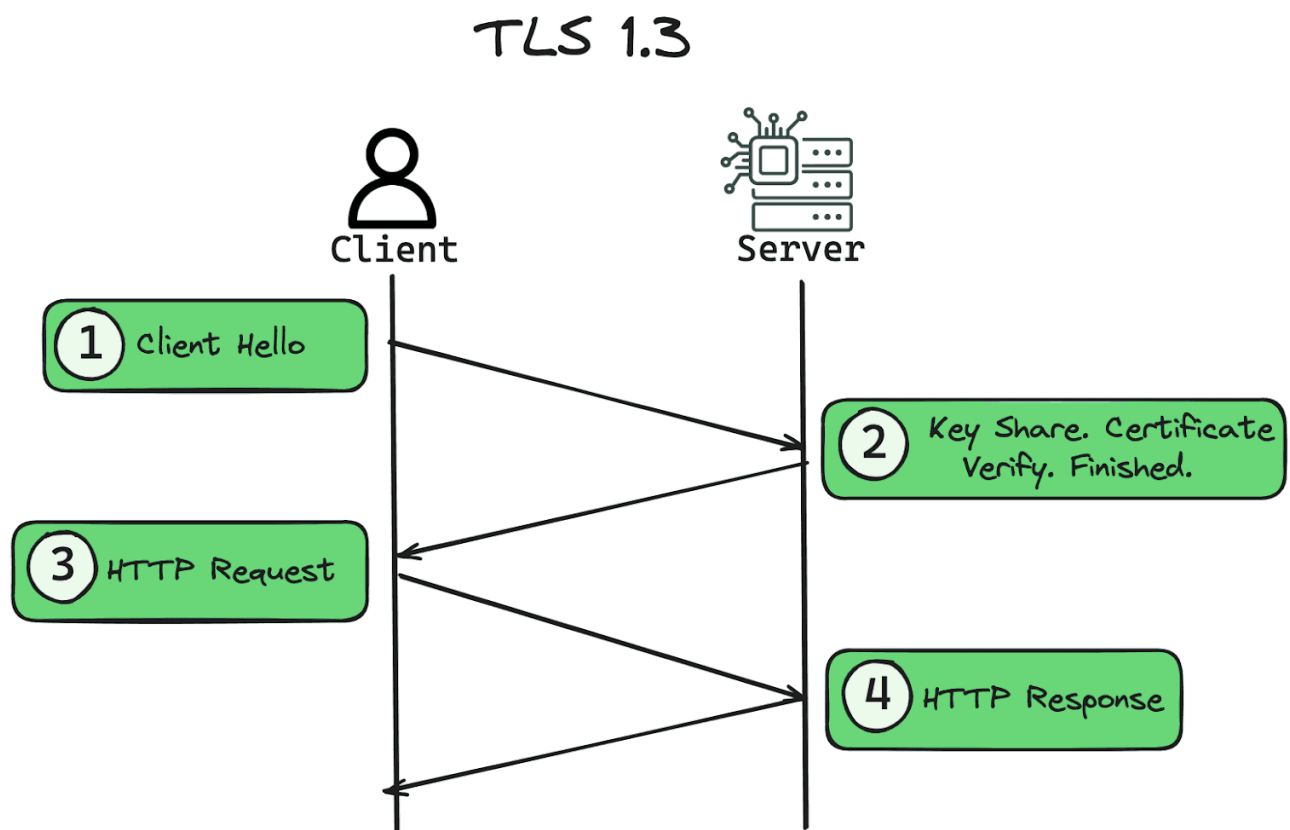
Overview of the Handshake

The TLS handshake is a dynamic back-and-forth process that establishes trust between two parties, paving the way for secure communications over the Internet. Within this procedure, both parties engage in a dialogue to authenticate each other, ensuring a foundation of trust. The protocol utilizes keys and encryption algorithms, negotiating the optimal set for the session at hand. This negotiation ensures that all data exchanged is encrypted and tailored to the security capabilities of both the client and the server.

Step-by-Step Breakdown of The TLS 1.3 Protocol

In this section, we examine TLS 1.3, the latest version of the protocol. Later, we explore TLS 1.2 to highlight improvements.

TLS 1.3



1. Initiation (Client Hello)

You know where their secure mailbox is, but you've never exchanged keys before. So you drop a note saying:

"Hi, I'm your friend, and I want to start a secure conversation. Here are the languages I can speak (types of encryption I can use), and here are some secrets (keys) I'm willing to use."

This is the "Client Hello" message, where your browser (client) reaches out to a website (server) to initiate a secure connection, suggesting encryption methods it supports.

2. Key Share, Certificate Verify, Finished

Key Share: Your friend receives your note and chooses a language (encryption method) both of you know. They also share their own secret (key) for encoding messages, similar to choosing a private language only both can understand.

Certificate Verify: To prove they are the rightful owner of the mailbox, your friend shows a badge (certificate) signed by a trusted authority.

Finished: After checking the badge, both of you agree on a shared secret (secure encryption key). This final agreement establishes the secure communication channel.

3. Start of the Meeting (HTTP Request)

Now that a secure channel is established, you write your actual letter (web request), put it in a sealed encrypted envelope, and send it through the mailbox. This represents the secure HTTP request sent by the browser.

4. Response (HTTP Response)

Your friend receives your letter, decrypts it, reads your request, and responds with the requested information. They encrypt the response and send it back. You decrypt it and read it.

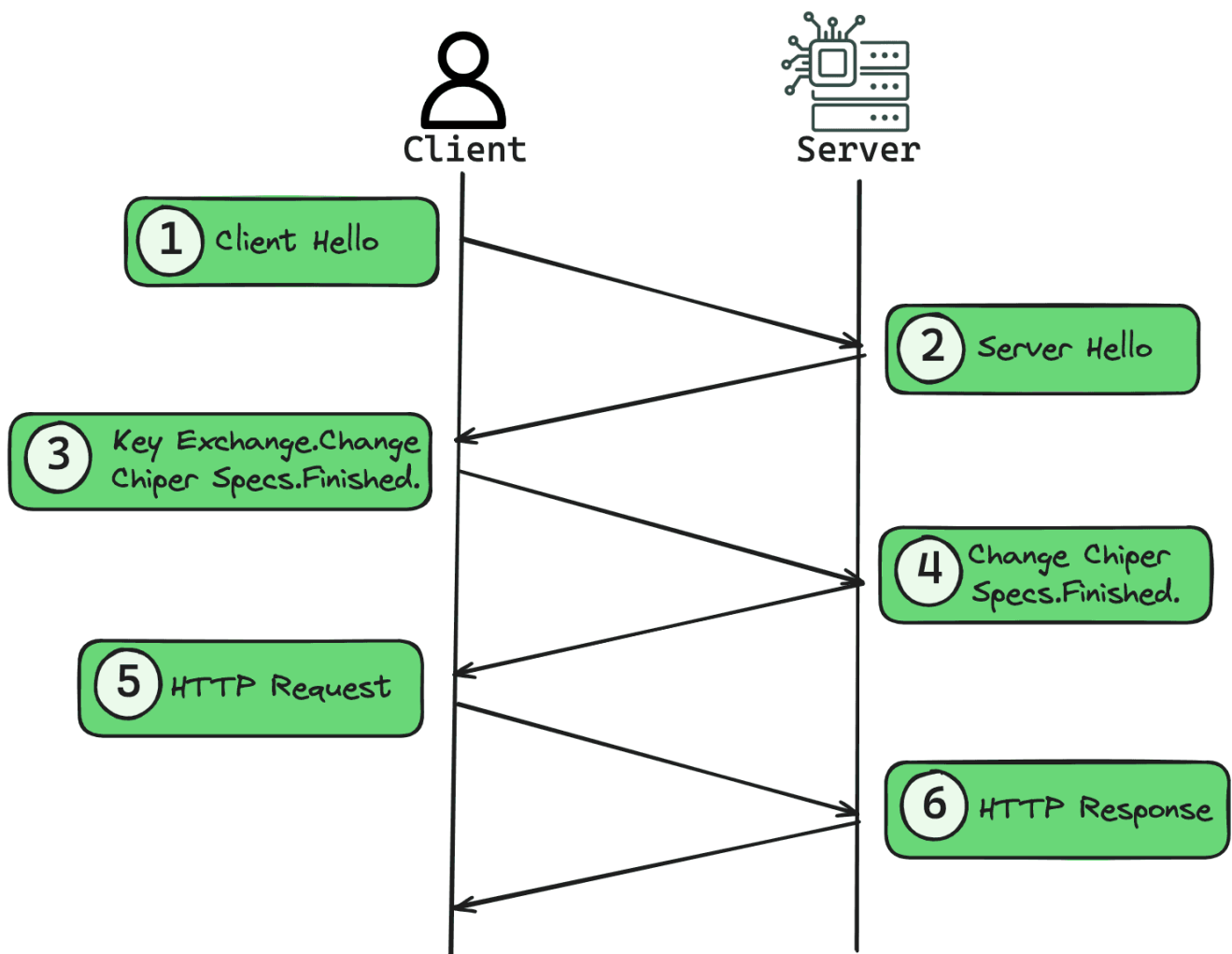
TLS 1.3 Handshake Optimization

TLS 1.3 introduced major improvements over TLS 1.2, mainly focusing on speed and security.

1. Fewer Round Trips

TLS 1.3 reduces handshake round trips to just one, compared to two in TLS 1.2. This is achieved by simplifying handshake messages.

TLS 1.2



2. Better Session Resumption

TLS 1.2 supports session resumption, but TLS 1.3 improves it with **0-RTT (Zero Round Trip Time)** data, allowing clients to send encrypted data immediately when resuming a session.

3. Improved Security

Forward Secrecy: TLS 1.3 requires ephemeral key exchange, ensuring each session uses unique encryption keys.

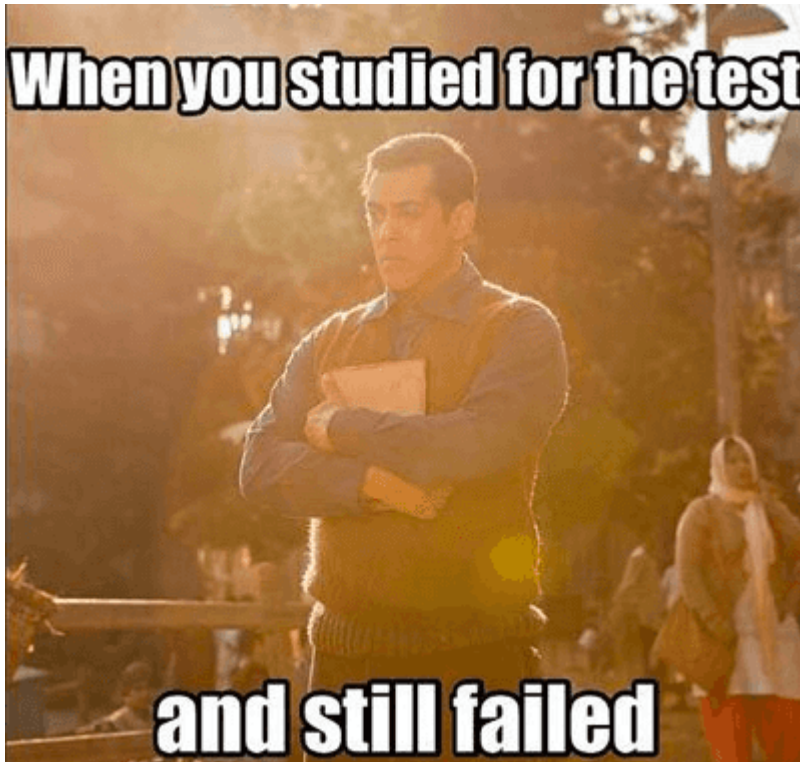
Modern Cryptography: TLS 1.3 removes outdated algorithms and enforces modern secure cipher suites.

4. Simplified Protocol

TLS 1.3 removes outdated features, reducing complexity and lowering the risk of misconfiguration and vulnerabilities.

Breaking the TLS Protocol and Its Future Trends

Even though TLS significantly improves internet security, no system is fully immune to attacks.

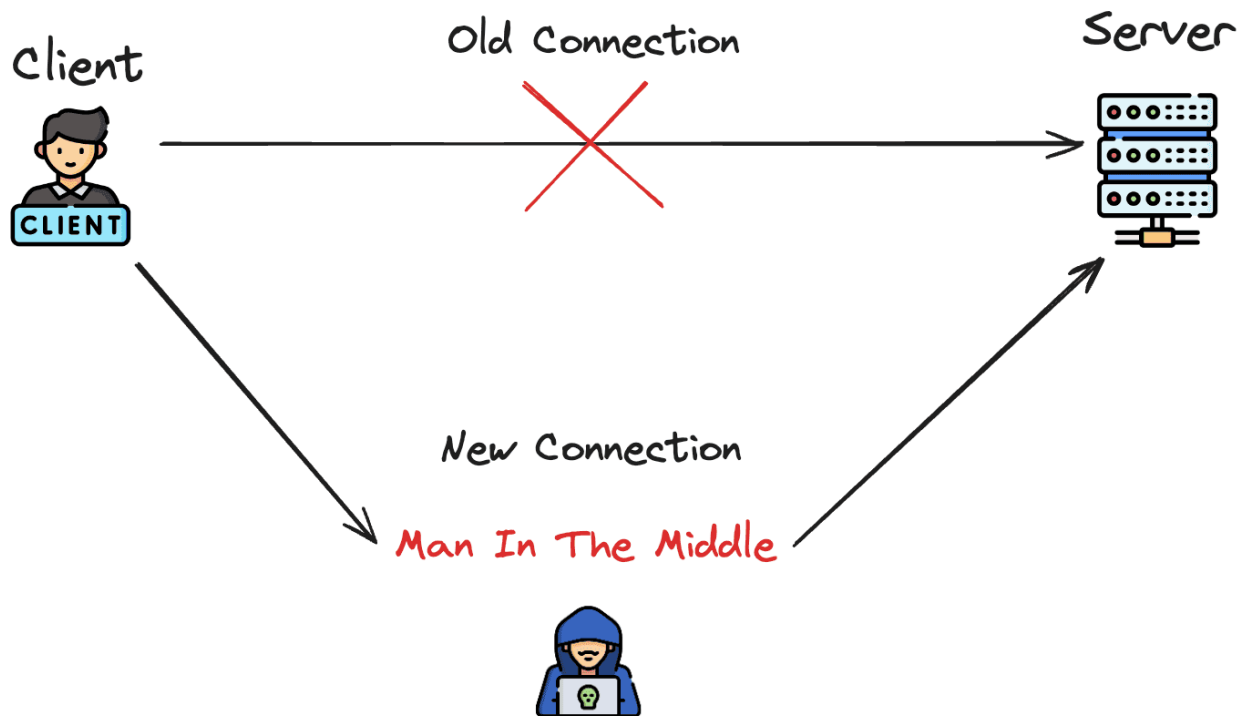


1. Man-in-the-Middle (MitM) Attacks

In a MitM attack, the attacker intercepts communication between client and server to eavesdrop or modify data. TLS reduces this risk using encryption and authentication, but weak configurations or compromised certificate authorities can still create vulnerabilities.

Attackers may position themselves between client and server using techniques such as ARP poisoning.

Man In The Middle Attack



They can then decrypt, modify, or inject malicious data into the communication.

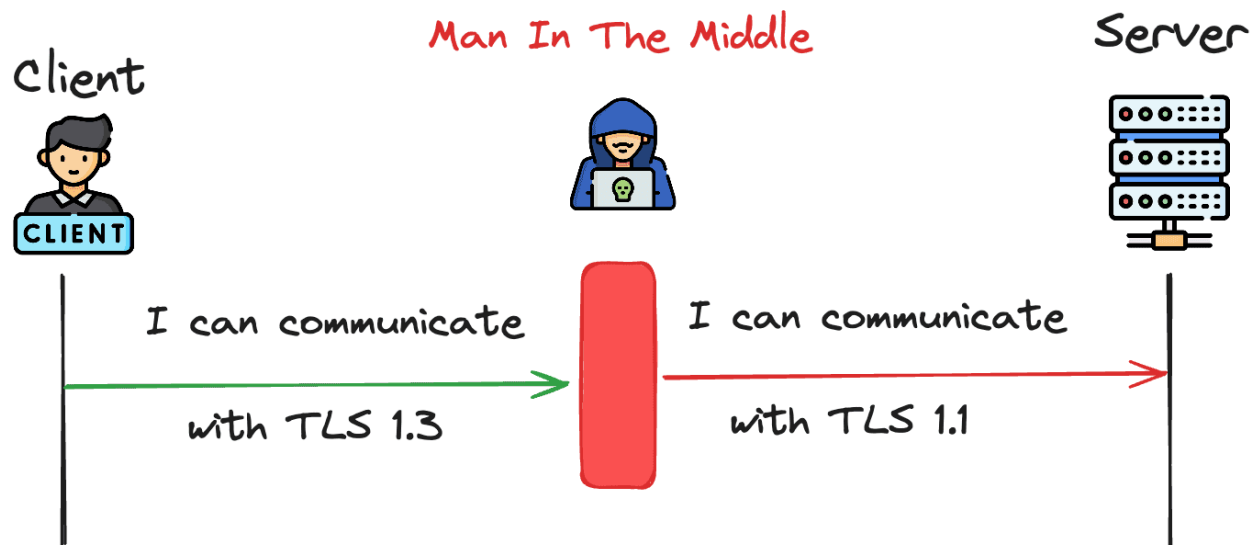
Proper TLS configuration and strong encryption reduce this risk.

2. Downgrade Attacks

Downgrade attacks force both parties to use older, insecure TLS versions.

The attacker intercepts the ClientHello message and modifies it to indicate weaker protocol support.

Downgrade Attack



The server then selects an older TLS version, exposing vulnerabilities that attackers can exploit.

How Can You Keep Your Web Project Secure

Using up-to-date TLS versions is essential for protecting authentication, credentials, and user sessions.

Best practices include:

- Disable outdated protocols such as SSL 2.0/3.0 and TLS 1.0/1.1
- Enforce HTTPS across all application pages
- Ensure all communications use TLS 1.3 when possible

Conclusion

The evolution from SSL to TLS 1.3 represents a major improvement in internet security. However, vulnerabilities still exist, requiring continuous improvement and proper configuration.

Developers must apply secure TLS configurations, and users must remain aware of security risks. Together, these practices strengthen the security of modern web communications.

6.7 Commands for TLS/HTTPS Analysis

```
# Check a website's TLS certificate
openssl s_client -connect google.com:443

# View certificate details
openssl s_client -connect google.com:443 | openssl x509 -text -noout
```

```
# Check supported TLS versions
nmap --script ssl-enum-ciphers -p 443 target.com

# Test TLS version and cipher support
nmap -sV --version-intensity 5 -p 443 target.com

# Check for weak SSL/TLS configurations
nmap --script ssl-heartbleed -p 443 target.com

# Curl with verbose TLS info
curl -v https://google.com 2>&1 | grep -A5 "TLS"

# Test specific TLS version
openssl s_client -tls1_2 -connect target.com:443
openssl s_client -tls1_3 -connect target.com:443
```

Module 6 — Review Questions

Q1. What are the three guarantees that HTTPS provides over plain HTTP? Explain each one.

Q2. In the TLS 1.2 handshake, what is the purpose of the **pre-master secret**? Who generates it and how is it securely transmitted?

Q3. Why is **TLS 1.3** considered more secure and faster than TLS 1.2?

Q4. A user connects to <http://banking-site.com> (no HTTPS). An attacker on the same Wi-Fi performs an **SSL stripping attack**. Describe step-by-step what happens.

Q5. What is a **Certificate Authority (CA)**? What is the risk if a CA is compromised? Can you name a real-world example of a CA compromise?

Q6. Using `openssl`, write the command to check the TLS certificate of univ-batna2.dz. What information would you expect to see?

Final Review and Wrap-Up

Complete Topic Map

```
Introduction to Cybersecurity
|
|— CIA Triad (Confidentiality, Integrity, Availability)
|— Threat Actors (White/Grey/Black Hat)
|— Cyber Kill Chain
|
|— OSI Model (7 Layers)
|   |— Physical → Data Link → Network → Transport
|   |— Session → Presentation → Application
```

- └─ TCP/IP Model (4 Layers)
 - └─ IPv4/IPv6 Addressing
 - └─ TCP Three-Way Handshake
 - └─ TCP vs UDP
- └─ Core Protocols
 - └─ ICMP (ping, traceroute)
 - └─ ARP (IP → MAC, spoofing)
 - └─ Port Numbers (Well-Known Ports)
- └─ DNS
 - └─ Resolution Process (Recursive → TLD → Auth)
 - └─ Record Types (A, AAAA, MX, CNAME, NS, PTR)
 - └─ Attacks (Spoofing, Hijacking, Zone Transfer)
- └─ HTTP/HTTPS
 - └─ HTTP Methods and Status Codes
 - └─ TLS 1.2 and TLS 1.3 Handshake
 - └─ Attacks (SSL Stripping, Heartbleed, POODLE)

Key Takeaways

1. **Every open port is a potential attack surface** — minimize services running on production systems.
 2. **HTTP is insecure** — always use HTTPS with modern TLS (1.2 or 1.3) for any sensitive communication.
 3. **DNS is foundational but fragile** — DNS poisoning can silently redirect users to malicious sites.
 4. **Reconnaissance is the first step of any attack** — Nmap is what attackers use; understanding it helps defenders.
 5. **The OSI model maps attacks to layers** — knowing which layer an attack targets helps select the right defense.
-

Final Comprehensive Questions

Q1. A user visits `http://mail.company.com` and enters their credentials. An attacker on the same LAN intercepts the traffic. Describe the full attack chain using the concepts from this course (ARP, MITM, HTTP, TLS).

Q2. You are hired as a penetration tester. Your first task is to map the internal network `10.0.0.0/24`. Write the complete sequence of Nmap commands you would use from initial host discovery to service version enumeration, and explain why you chose each command.

Q3. A web server returns `nmap -sV` result: `443/tcp open ssl/https Apache/2.4.29`. Search CVE databases and explain what you would investigate next.

Q4. Map the TLS handshake process to the OSI model — at which layer does each step operate? Which OSI layer is most relevant to TLS?

Q5. Draw and label a complete diagram showing: a client browser making an HTTPS request — include DNS resolution, TCP handshake, TLS handshake, and HTTP data exchange. Label which protocol is active at each step.

Recommended Resources

Resource	URL	Notes
Nmap Official Docs	https://nmap.org/docs.html	Complete reference
CVE Database	https://cve.mitre.org	Vulnerability lookup
Wireshark	https://www.wireshark.org	Packet capture analysis
OWASP Top 10	https://owasp.org/top10	Web application risks
TryHackMe	https://tryhackme.com	Hands-on practice
Hack The Box	https://hackthebox.com	CTF challenges
SSL Labs	https://ssllabs.com/ssltest	Test HTTPS config
DNS Lookup	https://dnschecker.org	DNS record lookup